# Programming Fundamentals

## Conditional Statements

Programming Fundamentals Team

SETU

2026-06-17

# Outline

# Outline

# 1. Flow of Control in a Program

Each program you write will typically have:

# 1. Flow of Control in a Program

Each program you write will typically have:

| | |
|---|---|
| **Sequence** | Things that will be done in a particular order |
| **Selection** | Things that will be done conditionally |
| **Iteration** | Things that will be done repetitively |

# 1. Flow of Control in a Program

For this slide deck, we will be covering the first two types of control flow: sequence and selection. We will cover iteration in a later slide deck.

# 1. Flow of Control in a Program

For this slide deck, we will be covering the first two types of control flow: sequence and selection. We will cover iteration in a later slide deck.

| | |
|---|---|
| **Sequence** | Things that will be done in a particular order |
| **Selection** | Things that will be done conditionally |
| **Iteration** | Things that will be done repetitively |

# 1. Flow of Control in a Program

For this slide deck, we will be covering the first two types of control flow: sequence and selection. We will cover iteration in a later slide deck.

| | |
|---|---|
| **Sequence** | Things that will be done in a particular order |
| **Selection** | Things that will be done conditionally |
| **Iteration** | Things that will be done repetitively |

Sequence is the default flow of control in a program. It simply means that the statements in a program will be executed in the order they are written.

# 1. Flow of Control in a Program

| | |
|---|---|
| **Sequence** | Things that will be done in a particular order |
| **Selection** | Things that will be done conditionally |
| **Iteration** | Things that will be done repetitively |

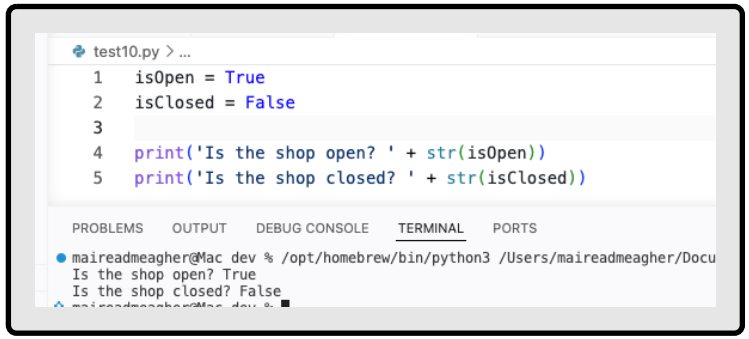
# 1. Flow of Control in a Program

| | |
|---|---|
| **Sequence** | Things that will be done in a particular order |
| **Selection** | Things that will be done conditionally |
| **Iteration** | Things that will be done repetitively |

- For the rest of this slide deck, we will be covering selection statements.
- Selection statements allow us to execute certain statements only if a certain condition is true.
- The most common selection statement is the if statement.

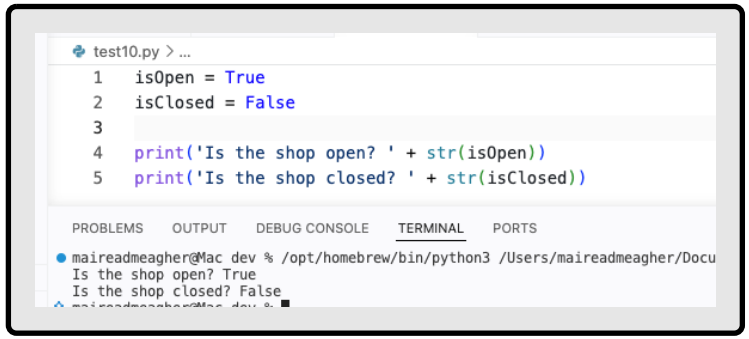
# Outline

# 2.1 Data types - Boolean




- Remember our discussion of data types? We will be using the data type called **Boolean** in selection statements.
- A Boolean is a data type that can only have two values: True or False.

# 2.1 Data types - Boolean




- Remember our discussion of data types? We will be using the data type called **Boolean** in selection statements.
- A Boolean is a data type that can only have two values: True or False.
- Booleans are often used in programming to represent the truth value of a condition or to control the flow of a program based on certain conditions.

# 2.2 Boolean Conditions

- A Boolean condition is an expression that evaluates to either True or False e.g.

```
x < 50
number > 0
```

# 2.2 Boolean Conditions

- A Boolean condition is an expression that evaluates to either True or False e.g.

```
x < 50
number > 0
```

- A conditional statement evaluates a Boolean condition and its result will determine which section of the code is executed.

# Outline

# 3. Relational Operators

| Operator | Syntax | Returns true if... |
|---|---|---|
| > | x > y | x is **greater** than y |
| >= | x >= y | x is **greater than or equal** to y |
| < | x < y | x is **less** than y |
| <= | x <= y | x is **less than or equal** to y |
| == | x == y | x and y are **equal** |
| != | x != y | x and y are **not equal** |

# 3. Relational Operators

| Operator | Syntax | Returns true if... |
|---|---|---|
| > | x > y | x is **greater** than y |
| >= | x >= y | x is **greater than or equal** to y |
| < | x < y | x is **less** than y |
| <= | x <= y | x is **less than or equal** to y |
| == | x == y | x and y are **equal** |
| != | x != y | x and y are **not equal** |

**= is an assignment operator and doesn't test for equality.**

**== is used to test for equality.**

# Outline

# 4.1 if statement syntax

```
if perform some boolean test :
    Do these statement(s) if the test is true
```

← actions if condition is true

# 4.1 if statement syntax

```
if perform some boolean test :
    Do these statement(s) if the test is true
```

← actions if condition is true

**NOTE:**

- Python relies on indentation to define scope in the code.
- Indentation is a whitespace at the beginning of the line.
- We saw this earlier when writing functions.

# 4.1 if statement syntax

Example of an if statement:

```
age = 25
if age >= 18:
    print("You are an adult.")
    print("You can vote.")
```

# 4.1 if statement syntax

Example of an if statement:

```
age = 25
if age >= 18:
    print("You are an adult.")
    print("You can vote.")
```

This will output:

```
You are an adult.
You can vote.
```

# 4.1 if statement syntax

Example of a sequence of if statements:

```
age = 18
if age > 18:
    print("You are an adult.")
    print("And you are over eighteen")

if age == 18:
    print("Congratulations on reaching
adulthood!")

if age < 18:
    print("You are a minor.")
```

# 4.1 if statement syntax

Example of a sequence of if statements:

```
age = 18
if age > 18:
    print("You are an adult.")
    print("And you are over eighteen")

if age == 18:
    print("Congratulations on reaching
adulthood!")

if age < 18:
    print("You are a minor.")
```

This will output:

```
Congratulations on reaching adulthood!
```

# 4.2 if-else statement syntax

```
if perform some boolean test :
    Do these statement(s) if the test
    is true
else:
    Do these statement(s) if the test
    is false
```

← actions if condition is **true**

← actions if condition is **false**

# 4.2 if-else statement syntax

```
if perform some boolean test :
    Do these statement(s) if the test
    is true
else:
    Do these statement(s) if the test
    is false
```

← actions if condition is **true**

← actions if condition is **false**

The `else` clause has **no condition** — it runs whenever the `if` condition is false. Only one branch (either `if` or `else`) will execute.

# 4.2 if-else statement syntax

Returning to the age example — we had two separate `if` statements:

```
# Two separate if statements
age = 20
if age >= 18:
    print("You are an adult.")
if age < 18:
    print("You are a minor.")
```

# 4.2 if-else statement syntax

Returning to the age example — we had two separate `if` statements:

```
# Two separate if statements
age = 20
if age >= 18:
    print("You are an adult.")
if age < 18:
    print("You are a minor.")
```

Simplified with if-else

```
# Simplified with if-else
age = 20
if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

# 4.2 if-else statement syntax

Returning to the age example — we had two separate `if` statements:

```
# Two separate if statements
age = 20
if age >= 18:
    print("You are an adult.")
if age < 18:
    print("You are a minor.")
```

Simplified with if-else

```
# Simplified with if-else
age = 20
if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

- The `if-else` version is cleaner — there are only two outcomes, so `else` handles the second case directly.

# 4.2 if-else statement syntax

A fresh example with `if-else`:

```
temperature = 22

if temperature > 25:
    print("It is warm outside.")
    print("Wear light clothing.")
else:
    print("It is cool outside.")
    print("Bring a jacket.")
```

# 4.2 if-else statement syntax

A fresh example with `if-else`:

```
temperature = 22

if temperature > 25:
    print("It is warm outside.")
    print("Wear light clothing.")
else:
    print("It is cool outside.")
    print("Bring a jacket.")
```

This will output:

```
It is cool outside.
Bring a jacket.
```

# 4.3 if-elif-else statement syntax

```
if condition1 :
    statements if condition1 is true
elif condition2 :
    statements if condition1 false AND condition2 true
else:
    statements if both conditions are false
```

# 4.3 if-elif-else statement syntax

```
if condition1 :
    statements if condition1 is true
elif condition2 :
    statements if condition1 false AND condition2 true
else:
    statements if both conditions are false
```

You can have as many `elif` clauses as you need. Only the **first** matching branch executes — the rest are skipped.

# 4.3 if-elif-else statement syntax

Example using `if-elif-else`:

```
score = 65

if score >= 80:
    print("Grade: Distinction")
elif score >= 60:
    print("Grade: Merit")
elif score >= 40:
    print("Grade: Pass")
else:
    print("Grade: Fail")
```

# 4.3 if-elif-else statement syntax

Example using `if-elif-else`:

```
score = 65

if score >= 80:
    print("Grade: Distinction")
elif score >= 60:
    print("Grade: Merit")
elif score >= 40:
    print("Grade: Pass")
else:
    print("Grade: Fail")
```

With `score = 65`, this outputs:

```
Grade: Merit
```

Try changing `score` and observe which branch runs:

- score = 90 ⟶ **Distinction**
- score = 70 ⟶ **Merit**
- score = 45 ⟶ **Pass**
- score = 30 ⟶ **Fail**

# Outline

# 5. Logical Operators

- Logical operators operate on Boolean values and produce a new Boolean value as a result.

# 5. Logical Operators

- Logical operators operate on Boolean values and produce a new Boolean value as a result.
- The three logical operators in Python are:

| Operator | Meaning | Example |
|---|---|---|
| `and` | True if **both** conditions are true | `age >= 18 and score > 50` |
| `or` | True if **at least one** condition is true | `day == "Sat" or day == "Sun"` |
| `not` | Reverses the Boolean value | `not is_raining` |

# 5.1 and operator

```
a and b
```

| a | b | a and b |
|---|---|---|
| True | True | **True** |
| False | True | **False** |
| True | False | **False** |
| False | False | **False** |

# 5.1 and operator

`a and b`

| a | b | a and b |
|---|---|---|
| True | True | **True** |
| False | True | **False** |
| True | False | **False** |
| False | False | **False** |

- Evaluates to **true** only if **both** a and b are true.
- It is false in all other cases.

```
age = 25
income = 35000

if age >= 18 and income >= 30000:
    print("Eligible for loan")
else:
    print("Not eligible")
```

# 5.2 or operator

```
a or b
```

| a | b | a or b |
|---|---|---|
| False | False | **False** |
| False | True | **True** |
| True | False | **True** |
| True | True | **True** |

# 5.2 or operator

`a or b`

| a | b | a or b |
|---|---|---|
| False | False | **False** |
| False | True | **True** |
| True | False | **True** |
| True | True | **True** |

- Evaluates to **true** if either a, b, or both are true.
- It is false only when **both** are false.

```
day = "Saturday"

if day == "Saturday" or day == "Sunday":
    print("It is the weekend!")
else:
    print("It is a weekday.")
```

# 5.3 not operator

```
not a
```

| a | not a |
|---|---|
| False | True |
| True | False |

# 5.3 not operator

`not a`

| a | not a |
|---|---|
| False | True |
| True | False |

- Reverses (negates) the Boolean value.
- True becomes False, and False becomes True.

```
is_raining = False

if not is_raining:
    print("Leave your umbrella at home.")
else:
    print("Don't forget your umbrella!")
```

# 5.4 Logical Operators — Quiz

Given these variables:

```
a = 5     b = 10     c = 7
```

What is the result of each Boolean expression?

| Q | Expression |
|---|---|
| Q1 | `(a > b) and (a < c)` |
| Q2 | `(a < b) or (c < a)` |
| Q3 | `not(b < a) and (c > b)` |

# 5.4 Logical Operators — Quiz

Answers:

```
a = 5     b = 10     c = 7
```

| Q | Evaluation | Result |
|---|---|---|
| Q1 | `(5 > 10)` is **False**. `(5 < 7)` is **True**. So `False and True` evaluates to **False**. | **False** |
| Q2 | `(5 < 10)` is **True**. With `or`, one true operand is enough — overall answer is **True**. | **True** |
| Q3 | `not(10 < 5)` = `not(False)` = **True**. `(7 > 10)` is **False**. So `True and False` evaluates to **False**. | **False** |

# Outline

# 6. Syntax Errors

As you write conditional statements, you will encounter Syntax Errors. Here are some common ones to watch out for.

# 6. Syntax Errors

As you write conditional statements, you will encounter Syntax Errors. Here are some common ones to watch out for.

**Learning tip:** Deliberately introduce these errors in VS Code, read the error message, and fix them. The more errors you encounter and fix, the faster you will recognise them in your own code.

# 6.1 Splitting a long Boolean condition

**Problem** — splitting a long condition across lines:

```
age = 25
income = 35000

if age >= 18 and
   income >= 30000:
    print("Eligible")
```

```
SyntaxError: invalid syntax
```

# 6.1 Splitting a long Boolean condition

**Problem** — splitting a long condition across lines:

```
age = 25
income = 35000

if age >= 18 and
   income >= 30000:
    print("Eligible")
```

```
SyntaxError: invalid syntax
```

**Fix** — wrap the full condition in parentheses:

```
age = 25
income = 35000

if (age >= 18 and
    income >= 30000):
    print("Eligible")
```

Parentheses allow the condition to span multiple lines.

✓

# 6.2 Missing indentation

**Problem** — body of `if` not indented:

```
age = 25
if age >= 18:
print("You are an adult.")
print("You can vote.")
```

```
IndentationError: expected an indented block
```

# 6.2 Missing indentation

**Problem** — body of `if` not indented:

```
age = 25
if age >= 18:
print("You are an adult.")
print("You can vote.")
```

```
IndentationError: expected an indented block
```

**Fix** — indent the body consistently:

```
age = 25
if age >= 18:
    print("You are an adult.")
    print("You can vote.")
```

Python uses indentation to define code blocks. ✓

# 6.3 else with a condition

**Problem** — adding a condition to `else`:

```
score = 65
if score >= 60:
    print("Pass")
else score < 60:
    print("Fail")
```

```
SyntaxError: invalid syntax
```

# 6.3 else with a condition

**Problem** — adding a condition to `else`:

```
score = 65
if score >= 60:
    print("Pass")
else score < 60:
    print("Fail")
```

```
SyntaxError: invalid syntax
```

**Fix** — use `elif` when you need a condition, or remove it from `else`:

```
score = 65
if score >= 60:
    print("Pass")
elif score < 60:    # elif has a condition
    print("Fail")

# OR simply:
if score >= 60:
    print("Pass")
else:               # else has no condition
    print("Fail")
```

# 6.4 elif without a matching if

**Problem** — `elif` indented inside the `if` body:

```
score = 65
if score >= 80:
    print("Distinction")
    elif score >= 60:
        print("Merit")
```

```
SyntaxError: invalid syntax
```

`elif` must be at the **same** indentation level as its `if`.

# 6.4 elif without a matching if

**Problem** — `elif` indented inside the `if` body:

```
score = 65
if score >= 80:
    print("Distinction")
    elif score >= 60:
        print("Merit")
```

```
SyntaxError: invalid syntax
```

`elif` must be at the **same** indentation level as its `if`.

**Fix** — align `elif` with `if`:

```
score = 65
if score >= 80:
    print("Distinction")
elif score >= 60:
    print("Merit")
elif score >= 40:
    print("Pass")
else:
    print("Fail")
```

`if`, `elif`, and `else` must all share the same indentation level. ✓

# Thanks for Watching - Any questions?